

07 — Arithmetic Intensity

Pré-requisito: [06-memory-as-cost-model.md](#)

Esta é a fórmula que todo o resto do módulo constrói até aqui pra você entender de verdade, não só decorar.

1. A fórmula

Arithmetic Intensity (AI) = FLOPs realizados / bytes movidos da/para memória

$$AI = \frac{\text{operações de ponto flutuante}}{\text{bytes lidos} + \text{bytes escritos}}$$

Unidade: FLOPs por byte. É **uma proporção**, não um valor absoluto de velocidade — ela mede "quanto cálculo eu consigo espremer de cada byte que precisei mover", exatamente a pergunta que o arquivo 06 preparou você pra fazer.

2. Calculando na mão: `vector_add`

```
void vector_add(const float* a, const float* b, float* out, int n) {  
    for (int i = 0; i < n; i++) {  
        out[i] = a[i] + b[i];  
    }  
}
```

Por elemento:

- **FLOPs:** 1 soma = 1 FLOP
- **Bytes lidos:** `a[i]` (4 bytes) + `b[i]` (4 bytes) = 8 bytes
- **Bytes escritos:** `out[i]` (4 bytes) = 4 bytes
- **Bytes totais:** 8 + 4 = 12 bytes

$AI(\text{vector_add}) = 1 \text{ FLOP} / 12 \text{ bytes} \approx 0.083 \text{ FLOPs/byte}$

Um número **muito baixo**. Isso significa: pra cada byte movido, você faz muito pouco cálculo. `vector_add` é, como você provavelmente já suspeitava depois de escrever ele hoje, extremamente memory-bound.

3. Calculando na mão: multiplicação de matriz (N×N)

```
// C = A * B, todas N x N
for (int i = 0; i < N; i++)
  for (int j = 0; j < N; j++)
    for (int k = 0; k < N; k++)
      C[i][j] += A[i][k] * B[k][j];
```

- **FLOPs totais:** para cada elemento de **C** (existem N^2 deles), o loop interno faz N multiplicações e N somas = $2N$ FLOPs. Total: $N^2 \times 2N = 2N^3$ FLOPs.
- **Bytes movidos** (versão ingênua, sem reuso de cache — o pior caso): cada elemento de **C[i][j]** lê uma linha inteira de **A** (N floats) e uma coluna inteira de **B** (N floats) do zero. Bytes lidos $\approx N^2 \times 2N \times 4$ bytes (ignorando qualquer reuso de cache, que na prática existe, mas vamos com o caso ingênuo pra ver o número bruto primeiro).

$$AI(\text{matmul, ingênuo}) = 2N^3 \text{ FLOPs} / (2N^3 \times 4 \text{ bytes}) = 1/4 = 0.25 \text{ FLOPs/byte}$$

Ainda baixo — **mas isso é enganoso**, e é exatamente por isso que matmul é um dos casos mais estudados em otimização de kernel: com **reuso de cache/shared memory** bem feito (tiling — técnica que reaparece no seu roadmap na Phase 3), você reduz drasticamente os bytes movidos sem mudar o número de FLOPs, e a **AI efetiva** sobe muito. É por isso que matmul bem otimizado é compute-bound, mas matmul ingênuo é memory-bound — **a mesma operação matemática**, dois regimes completamente diferentes, só por causa de como a memória é acessada.

PERGUNTA RÁPIDA 1: se você reescrever o matmul pra reusar cada linha de **A** carregada uma vez em cache para calcular **vários** elementos de **C** (em vez de recarregar do zero pra cada elemento), o que acontece com o numerador e o denominador da fórmula de AI?

► Resposta

4. Calculando na mão: uma redução (soma de todos os elementos)

```
float soma_total(const float* arr, int n) {
  float soma = 0;
  for (int i = 0; i < n; i++) soma += arr[i];
  return soma;
}
```

- **FLOPs:** n somas = n FLOPs
- **Bytes lidos:** n floats $\times$ 4 bytes = $4n$ bytes (a escrita final é irrelevante, um único float no final, não escala com n)

$$AI(\text{redução}) = n / 4n = 0.25 \text{ FLOPs/byte}$$

Igual à AI do matmul ingênuo, por coincidência de números — mas o ponto real é: **quase toda operação "simples" (soma, escala, produto elemento-a-elemento) tem AI baixíssima**, porque o numerador cresce linear com n e o denominador também cresce linear com n , na mesma proporção — a razão nunca escapa de valores pequenos, seja qual for n .

5. Como calcular pra QUALQUER operação: o método geral

1. Conte quantos FLOPs a operação faz, por elemento (ou no total).
2. Conte quantos bytes precisam ser lidos DA memória, por elemento (ou no total).
3. Conte quantos bytes precisam ser escritos PARA a memória, por elemento (ou no total).
4. $AI = \text{FLOPs} / (\text{bytes lidos} + \text{bytes escritos})$

Cuidado com uma armadilha comum: "bytes movidos" não é o tamanho do array — é quantos bytes **de fato** entram e saem da unidade de cálculo, contando reuso de cache quando ele existe de verdade. É por isso que a mesma operação matemática pode ter AI diferente dependendo de como o código está escrito — a fórmula mede o **código**, não só a matemática abstrata.

PERGUNTA RÁPIDA 2: calcule a AI de $y = a*x + b$ (uma operação escalar "axpy" — a e b são escalares fixos, x e y são arrays de n floats).

► Resposta

6. Uma ressalva importante: AI algorítmica vs AI de hardware

Tudo que você calculou até aqui é o que se chama **AI algorítmica** (ou "AI efetiva"): contada a partir da matemática do algoritmo e de quantos bytes o **código-fonte** parece exigir, na mão, no papel. Isso já é extremamente útil — é o que te diz, de antemão, se vale a pena otimizar cálculo ou memória, sem precisar rodar nada ainda.

Mas existe uma segunda AI, medida de forma completamente diferente: a **AI de hardware**, que conta o tráfego **real** que efetivamente atravessa um nível específico da hierarquia de memória (arquivo 04) — L1, L2, ou DRAM — durante a execução real, medida por contadores de hardware (performance counters), não inferida do código.

Por que as duas podem ser diferentes números: o hardware pode mover mais (ou, com bom reuso de cache, efetivamente menos "de novo") bytes do que o algoritmo pede no papel — cache line inteira sendo buscada quando só um float foi pedido (arquivo 04), reuso de dado que já estava em L1 nunca "contando" como tráfego de DRAM de novo, transações de memória agrupando requisições de formas que a contagem ingênua não prevê. O matmul do exemplo acima é o caso mais claro: a versão "ingênua" tem uma AI algorítmica baixa (≈ 0.25), mas a **AI de DRAM real**, medida com contadores de hardware, pode ser bem mais alta se o cache já está absorvendo boa parte do reuso sem você ter programado isso explicitamente — e o oposto também acontece, quando o padrão de acesso é ruim o suficiente pra gerar **mais** tráfego real do que a contagem ingênua sugere (por exemplo, lendo uma cache line inteira de 64 bytes pra aproveitar só 4).

A consequência prática: quando você começar a usar Nsight Compute (Phase 4 do seu roadmap), ele reporta roofline **hierárquico** — tetos diferentes pra L1, L2 e memória de dispositivo, cada nível com sua própria AI medida separadamente. "Otimizei e o tráfego de DRAM caiu, mas o tráfego de L1 subiu" é um resultado real e

comum, não uma contradição — os dois números medem coisas diferentes. Todo cálculo de AI que você fizer na mão, daqui pra frente, vale deixar explícito: **"esta é a AI algorítmica — uma estimativa de antemão, não uma medição de tráfego real de um nível específico da memória."** As duas são ferramentas diferentes, usadas em momentos diferentes do fluxo de trabalho: AI algorítmica antes de medir (pra decidir onde vale a pena focar), AI de hardware depois de medir com profiler (pra confirmar ou corrigir a expectativa).

7. Por que isso importa para CUDA

Arithmetic intensity é o número que decide se otimizar um kernel significa "reduzir FLOPs" ou "reduzir tráfego de memória" — e escolher a estratégia errada é desperdício de esforço garantido. Todo kernel real que você for otimizar depois (incluindo os do North Star do seu lab — normalização é notoriamente baixa AI) começa com esse cálculo, na mão, antes de qualquer profiling. É literalmente o primeiro passo do fluxo de trabalho descrito no `KERNEL_WORKFLOW.md` do seu próprio repositório.

8. Erros comuns

- Contar "bytes do array" em vez de "bytes efetivamente movidos considerando reuso" — superestima o denominador, subestima a AI real de código bem escrito.
- Esquecer de contar tanto leitura quanto escrita — só contar leitura, ou só escrita, subestima o denominador.
- Achar que AI é uma propriedade fixa de "essa operação matemática" — é uma propriedade do **código**, e muda com técnica de implementação (tiling, fusão, etc.), como o exemplo do matmul mostrou.
- Confundir AI algorítmica (calculada na mão, a partir da matemática do código) com AI de hardware (medida com contadores reais, num nível específico da hierarquia de memória) — são números diferentes, medidos de formas diferentes, úteis em momentos diferentes do fluxo de trabalho (seção 6).

PREDICT BEFORE RUNNING (cálculo na mão)

Calcule a arithmetic intensity de cada operação abaixo, mostrando FLOPs e bytes:

1. `out[i] = a[i]` (uma cópia simples, sem operação matemática nenhuma)
2. `out[i] = a[i] * a[i]` (elevar ao quadrado, um único array de entrada)
3. Uma operação fundida: `out[i] = (a[i] + b[i]) * c[i]` (lê 3 arrays, escreve 1)

► Explicação